

Fundamentals of

Artificial Intelligence

A comprehensive journey through AI concepts, techniques, and intelligent systems

Introduction • Knowledge Representation • Search • Agents

Course Overview

Topics Covered

01 Introduction to AI

02 AI Representation & Techniques

03 Knowledge Representation

04 Knowledge Base Systems

05 State Space & Production Systems

06 Problem Characteristics

07 Intelligent Agents

08 Problem Solving & Formulation

01

Introduction

to Artificial Intelligence

What is AI? • History • Goals • Applications



What is Artificial Intelligence?

Four Classical Definitions

Thinking Humanly

Systems that think like humans. Cognitive modeling approach — mirroring human thought processes.

Acting Humanly

Systems that act like humans. Turing Test approach — behavior indistinguishable from humans.

Thinking Rationally

Systems that think rationally. Laws of thought approach — using logic and formal reasoning.

Acting Rationally

Systems that act rationally. Rational agent approach — maximizing goal achievement.

History of Artificial Intelligence

Key Milestones & Eras

- **1950** Turing Test proposed by Alan Turing
- **1956** Dartmouth Conference — AI field born
- **1965** DENDRAL — first expert system developed
- **1986** Backpropagation revival for neural networks
- **1997** Deep Blue defeats chess world champion
- **2012** Deep Learning breakthrough — AlexNet on ImageNet
- **2016** AlphaGo defeats world Go champion
- **2023** Large Language Models — GPT era

AI Applications Today

AI is transforming every domain

Healthcare

Disease diagnosis, drug discovery, medical imaging

Natural Language

Machine translation, chatbots, sentiment analysis

Computer Vision

Object detection, facial recognition, autonomous cars

Robotics

Industrial automation, surgical robots, drones

Finance

Fraud detection, algorithmic trading, credit scoring

Game Playing

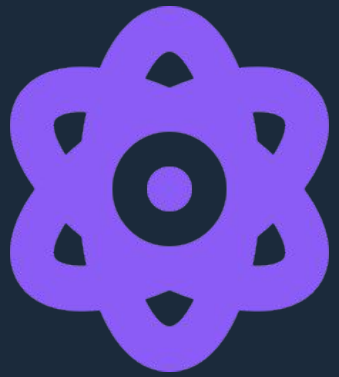
Chess, Go, video games — superhuman performance

02

AI Representation

Non-AI vs AI Techniques

Algorithms • Heuristics • Learning • Reasoning



Non-AI Techniques

Traditional algorithmic approaches to problem solving

Conventional Algorithms

Step-by-step procedures with deterministic outputs. No learning involved.

Statistical Methods

Pattern finding using probability and mathematical models.

Simulation

Modeling real-world systems to predict behavior.

Optimization Techniques

Linear programming, gradient descent, operations research.

Expert Manual Systems

Human-coded rule systems based on domain knowledge.

Key Characteristics

- ✓ Deterministic behavior
- ✓ Fixed rules / logic
- ✓ No self-improvement
- ✓ Explicit programming
- ✓ Predictable output
- ✓ Limited adaptability

AI Techniques

Intelligent problem-solving approaches

Search Algorithms

BFS, DFS, A*, heuristic search methods for exploring state spaces

Machine Learning

Supervised, unsupervised, and reinforcement learning paradigms

Knowledge Representation

Logic, semantic nets, frames, ontologies to encode knowledge

Reasoning & Planning

Inference engines, forward/backward chaining, automated planning

Neural Networks

Perceptrons, deep networks — inspired by the human brain

Fuzzy Logic

Handling imprecise and uncertain information mathematically

Non-AI vs AI Techniques — Comparison

Aspect	Non-AI Techniques	AI Techniques
Problem Solving	Explicit algorithmic steps	Search, heuristics, reasoning
Adaptability	Fixed, no learning	Learns and improves from data
Knowledge	Hard-coded logic	Represented and inferred
Uncertainty	Cannot handle uncertainty	Handles ambiguity & fuzzy data
Scalability	Degrades with problem complexity	Scales with intelligent methods
Examples	Sorting, calculators, GPS	Chatbots, chess AI, image recognition

03

Representation

of Knowledge

Logic • Semantic Nets • Frames • Rules • Ontologies



What is Knowledge Representation?

Knowledge Representation (KR) is the area of AI that studies how to formally encode information about the world so that a computer system can use it to solve complex tasks. It provides the vocabulary for describing the world and the inference mechanisms to derive new knowledge.

Propositional Logic

Simple true/false statements. $P \land Q \rightarrow R$

Predicate Logic

Quantified statements — $\forall x$, $\exists x$ predicates

Semantic Networks

Graph-based — nodes (concepts) + arcs (relations)

Frames

Structured records with slots and values (OOP-like)

Production Rules

IF condition THEN action — rule-based systems

Ontologies

Formal vocabularies of a domain with axioms

Logic-Based Knowledge Representation

Using formal logic to represent facts and rules

Propositional Logic

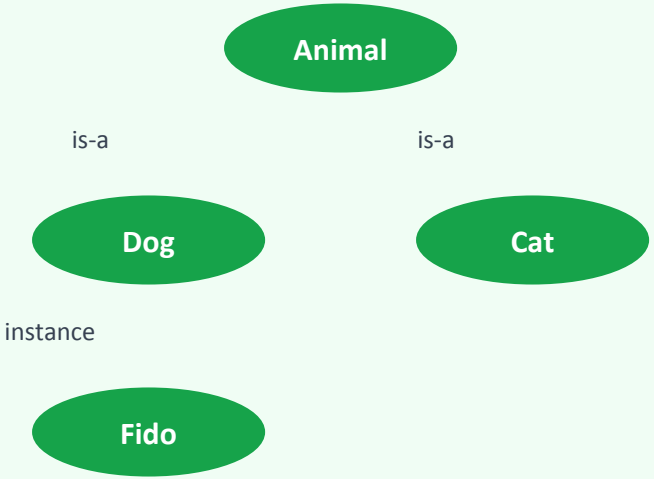
- Deals with propositions (true/false)
- Logical connectives: $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT), $\rightarrow$ (IF-THEN)
- Example: P = 'It is raining', Q = 'Ground is wet' $P \rightarrow Q$
- Limited — cannot express general facts

First-Order Predicate Logic

- Extends propositional logic with quantifiers
- Universal: $\forall x$ Human(x) $\rightarrow$ Mortal(x)
- Existential: $\exists x$ Parent(x, John)
- Supports objects, relations, functions

Semantic Networks & Frames

Semantic Network



- Graph structure with nodes (concepts) and arcs (relations)
- Supports inheritance, IS-A hierarchies
- Intuitive visual representation

Frames

Frame: Car

Slot	Value
Color	Red
Make	Toyota
Speed	120 km/h
Fuel	Petrol
AKO	Vehicle

Frames package related slots/values for an object — similar to objects in OOP

04

Knowledge Base

Systems

Expert Systems • Inference • Forward & Backward Chaining



Knowledge Base Systems

Architecture and Components

Knowledge Base

Facts + Rules

Working Memory

Current context

Inference Engine

Reasoning core

User Interface

Query/Explanation

Explanation Facil.

Why/How answers

Knowledge Acq.

Learning from expert

Knowledge Base Systems separate knowledge (what to know) from inference (how to reason) — the hallmark of expert systems like MYCIN, DENDRAL, and XCON.

Inference: Forward & Backward Chaining

Forward Chaining

Data-driven reasoning

- Start with known facts (data)
- Apply matching rules repeatedly
- Generate new facts until goal reached
- Example: Symptom → Disease diagnosis

Facts → Rules → Conclusions

Backward Chaining

Goal-driven reasoning

- Start with the hypothesis (goal)
- Work backward to find supporting facts
- Verify sub-goals recursively
- Example: Medical diagnosis from hypothesis

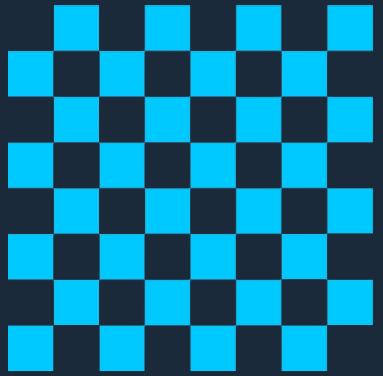
Goal → Rules → Supporting Facts

05

State Space Search

& Production Systems

BFS • DFS • A • Heuristics • Rules*



State Space Search

Exploring all possible states to find a solution

State

A snapshot of the world at a particular point in time. E.g., board configuration in chess.

Initial State

The starting configuration of the problem before any actions are taken.

Goal State

The desired end configuration that the search aims to reach.

Operators / Actions

Transitions that move from one state to another. E.g., move a tile left/right.

State Space

The complete set of all states reachable from the initial state via operators.

Path Cost

The cost associated with reaching a state — guides optimal search algorithms.

Search Strategies

Uninformed vs Informed Search

Algorithm	Type	Properties	Complexity
BFS (Breadth-First)	Uninformed	Complete, Optimal for unit cost	O(b^d)
DFS (Depth-First)	Uninformed	Incomplete (infinite loops), Not optimal	O(b^m)
UCS (Uniform Cost)	Uninformed	Complete, Optimal for path cost	O(b^(1+C*/ε))
Greedy Best-First	Informed	Not complete, Not optimal	O(b^m)
A* Search	Informed	Complete, Optimal (admissible h)	O(b^d)
IDA* Search	Informed	Complete, Optimal, Memory efficient	O(b^d)

Production Systems

Rule-based computational framework for AI problem solving

Production Memory

Rule Set

A set of IF-THEN rules called productions. Each rule has a condition (LHS) and an action (RHS).

Example: IF animal has feathers THEN it is a bird

Working Memory

Current State

A data structure that holds the current state of the world — all currently known facts and context.

Acts as a blackboard for rule matching.

Inference Engine

Control Cycle

The MATCH-SELECT-ACT cycle:
1. MATCH rules against working memory
2. SELECT a rule (conflict resolution)
3. FIRE the rule, update working memory

06

Problem

Characteristics

Decomposability • Reversibility • Predictability • Clarity



Characteristics of AI Problems

Properties that determine search strategy and solution approach

Decomposable?	✓ Can be broken into sub-problems	✗ Must be solved as a whole
Solution Steps Ignorable / Undoable?	✓ Backup is possible (reversible)	✗ Irrecoverable (chess moves)
Predictable?	✓ Certain outcomes (deterministic)	✗ Stochastic / uncertain outcomes
Good Solution Absolute / Relative?	✓ Any solution acceptable	✗ Best solution required (optimal)
Knowledge Required?	✓ Adequate knowledge available	✗ Requires learning / discovery

Types of Production Systems

Classification based on reversibility and predictability

M — Monotonic Production Systems

Actions do not remove existing working memory elements. Solution is reachable in all cases. Safe to apply rules independently.
Example: Mathematical theorem proving.

N — Non-Monotonic Production Systems

Actions may add or remove working memory elements. Require careful ordering of rules.
Example: Belief revision, planning systems.

P — Partially Commutative Systems

Order of rule application matters for some but not all rules. Mix of reversible and irreversible steps.
Example: Blocks world problem.

C — Commutative Production Systems

Any order of rule application leads to the same final result. Maximally flexible; no ordering constraints.
Example: Chemical equation balancing.

Control Strategies in Production Systems

How to select which rule to fire when multiple rules match

Recency

Prefer rules that match the most recently added working memory elements. Mimics human short-term focus.

Specificity

Prefer rules with more conditions on the left-hand side — more specific rules are given priority.

Rule Ordering

Apply rules in the order they are listed in the rule base. Simple but inflexible.

MEA (Means-Ends Analysis)

Select rules that reduce the difference between current state and goal state. Used in GPS.

Breadth-First

Fire all applicable rules at each step before moving to next level. Ensures completeness.

Depth-First

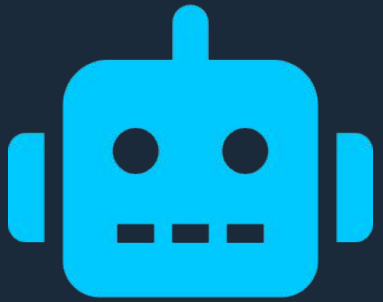
Follow one rule chain as far as possible before backtracking. Memory-efficient but may miss solutions.

07

Intelligent Agents

& Environments

Rationality • PEAS • Environment Types • Agent Architectures



What is an Intelligent Agent?

An agent is anything that perceives its environment through sensors and acts upon that environment through actuators. An intelligent agent does this in a way that maximizes its expected performance measure.

PEAS Framework

P	**Performance Measure**	Score, accuracy, profit
E	**Environment**	Physical/virtual world
A	**Actuators**	Motors, display, speech
S	**Sensors**	Camera, keyboard, GPS

Agent Diagram



Concept of Rationality

A rational agent is one that does the right thing — selects actions that maximize its expected performance measure, given the information available from its perceptions.

Performance Measure

The criterion that defines success. Must be defined externally to the agent — e.g., distance traveled, tasks completed correctly.

Prior Knowledge

What the agent knows about the environment before starting — built-in or learned knowledge.

Percept Sequence

Everything the agent has perceived so far. Rational action depends on the full percept history.

Available Actions

The set of all actions the agent can perform. Rationality requires choosing the best available action.

The Nature of Environments

Environment properties determine the complexity of the agent design

Fully Observable	**Partially Observable**	Can the agent see the entire state? Chess (full) vs Poker (partial)
Deterministic	**Stochastic**	Is the next state fully determined by current state and action?
Episodic	**Sequential**	Is each percept-action episode independent? Episodic is simpler.
Static	**Dynamic**	Does the environment change while the agent deliberates?
Discrete	**Continuous**	Finite number of percepts/actions (chess) vs infinite (driving)
Single Agent	**Multi-Agent**	One agent or multiple agents? Cooperative or competitive?

Structure of Agents

Agent = Architecture + Program

SR — **Simple Reflex Agent**

Acts based only on current percept. Uses condition-action rules. No memory.

MB — **Model-Based Reflex Agent**

Maintains internal state model of world. Handles partial observability.

GB — **Goal-Based Agent**

Has explicit goals. Plans actions to achieve goals. More flexible.

UB — **Utility-Based Agent**

Uses utility function to rank desirable states. Optimal performance measure.

LA — **Learning Agent**

Improves performance through experience. Has learning element and performance element.

Agent Architectures — Deep Dive

Simple Reflex vs Model-Based vs Goal-Based vs Utility-Based

Feature	Simple Reflex	Model-Based	Goal-Based	Utility-Based
Memory	None	Internal State	Goals + State	Utility + State
Decision Basis	Current percept	World model	Goal testing	Utility function
Handles Partial Obs	No	Yes	Yes	Yes
Optimal Action	No	No	No	Yes
Complexity	Low	Medium	Medium-High	High
Example	Thermostat	Robot navigator	Chess program	Self-driving car

Learning Agent Architecture

Self-improving agents through experience

Performance Element

Selects actions based on current knowledge. This is the 'agent' that acts in the environment.

Learning Element

Makes improvements to knowledge based on feedback from the critic. Core of adaptation.

Critic

Evaluates how well the agent is doing with respect to performance standard. Provides feedback.

Problem Generator

Suggests actions that will lead to new and informative experiences for learning.

Performance Standard → Critic → Learning Element → Knowledge → Performance Element → Actions

08

Problem Solving

Agents & Formulation

Goal Formulation • Problem Definition • Search • Execution



Problem Solving Agents

Goal-directed agents that plan sequences of actions

1

Goal Formulation

Define what the agent wants to achieve. Organize behavior around goals.

2

Problem Formulation

Decide what actions and states to consider given the goal.

3

Search

Find a sequence of actions that lead to goal state before executing.

4

Execute

Carry out the found solution. Blind execution of the action sequence.

Problem Formulation

Formal components of a well-defined problem

Initial State

The starting state s_0 from which the agent begins. E.g., initial board in 8-puzzle.

Actions

ACTIONS(s): set of actions available in state s. E.g., Up/Down/Left/Right for tile.

Transition Model

RESULT(s, a): describes what state results from action a in state s.

Goal Test

GOAL-TEST(s): whether state s is a goal state. Sometimes just one goal.

Path Cost Function

g(n): assigns numeric cost to each path. Step costs add up. Optimal search minimizes g.

Example: The 8-Puzzle Problem

A classic AI problem formulation example

Initial State

7	2	4
5		6
8	3	1

Goal State

1	2	3
4	5	6
7	8	

Problem Formulation

States:	All 9!/2 = 181,440 reachable positions
Initial:	Any tile arrangement
Actions:	Move blank U/D/L/R
Goal:	Tiles in order 1-8
Cost:	1 per tile move

This has 181,440 reachable states. A finds optimal in ~30 moves.*

Heuristic Functions in Search

Guiding search toward the goal efficiently

A heuristic function h(n) estimates the cost from node n to the nearest goal. An admissible heuristic never overestimates the true cost — this guarantees A* finds the optimal solution.

Manhattan Distance $h(n) = \Sigma\ |x_i - x_{goal}| + |y_i - y_{goal}|$

8-puzzle: sum of row+column distances of misplaced tiles

Misplaced Tiles h(n) = number of tiles not in goal position

Simpler to compute but less informative than Manhattan

Euclidean Distance $h(n) = \sqrt{((x-x_g)^2 + (y-y_g)^2)}$

Route-finding problems on maps — straight-line distance

Pattern Databases h(n) = precomputed exact cost of subproblem

Highly informed; used in 15-puzzle & combinatorial search

AI Concepts — Summary Matrix

Quick reference across all key topics

Topic	Key Concept	Real-World Example
Knowledge Representation	Encoding facts, rules, logic	Medical diagnosis rules
State Space Search	Explore states to find goal	GPS navigation, chess AI
Production Systems	IF-THEN rules + inference	CLIPS, DROOLS, expert systems
Knowledge Base Systems	Separate knowledge+inference	MYCIN diagnosis system
Intelligent Agents	Perceive → decide → act	Self-driving cars, Siri
Problem Formulation	States, actions, costs, goal	Route planning, scheduling
Heuristic Search	A*, h(n) guides to goal	Pathfinding in robotics

Key Takeaways

What we've learned about Artificial Intelligence

01 AI encompasses reasoning, learning, perception, and action — four key functional areas.

02 Knowledge representation is the foundation — how we encode the world determines how we reason about it.

03 Search is central to AI problem solving — uninformed vs informed search determine efficiency.

04 Production systems provide modular, maintainable AI via IF-THEN rules and inference engines.

05 Intelligent agents perceive, decide, and act — PEAS defines their interaction with the environment.

06 Rational agents maximize expected performance — the cornerstone of modern AI system design.

07 Problem formulation determines solution quality — good problem definition is half the solution.



Thank You

Fundamentals of Artificial Intelligence

Introduction • Representation • Knowledge Base • Search • Agents • Problem Solving

AI Education Series